Autoresearch Loops:
Causal Study of the Instruction File

Christian Block
Research project with Prof. Ramesh

August 31, 2026

1 Introduction

In an autoresearch loop, a language model does the experimenting. It is given a goal, some code it may edit, and an executor that runs a trial and hands back a number. From there it works on its own: propose a change, run it, read the result, decide what to try next, and repeat until the budget runs out. Karpathy's `autoresearch` [5] fixed the current meaning of the term when it appeared in March 2026, and it has been forked several thousand times since.

The architecture is a contract between three files, each with one owner. `prepare.py` prepares the data and defines the score. Nobody may touch it, so every experiment is scored the same way. `train.py` holds the model and the training loop, and only the agent edits it. `program.md` holds the instructions, and only the human edits it. Every candidate gets five minutes of wall clock and one number, validation bits per byte, so trials cost the same and compare directly. The loop ratchets through git: commit a change, train, keep the commit if the score improved, otherwise `git reset` back [2]. The repository only ever moves forward.

What the system does not contain is any orchestration code. There is no scheduler and no driver program. The nine-step experiment cycle exists only as English prose inside `program.md`, and the model is the execution layer. So `program.md` is not configuration handed to a controller. It is the controller.

1.1 Problem statement

These loops work. Why they behave as they do is not known. The AI Scientist [6] and Coscientist [3] are judged end to end, on whether the pipeline produced something acceptable, and benchmarks like MLAgentBench [4] report success rates over task suites. Industry reports the same shape of result. An autoresearch run against a production template engine cut parse-and-render time by 53% over about one hundred and twenty experiments, a gain its own author called probably overfitted and which was never merged [1]. All of this shows a system can work. None of it says which part of the instruction file did the work.

The obstacle is confounding. An instruction file settles several things at once: what counts as success, how widely the agent may search, when it must stop, how it must report, where to spend effort. Compare two differently worded files and every sentence differs, so nothing is identified. Work on prompt sensitivity makes the problem sharper. Formatting choices that preserve meaning can move accuracy by 76 points [7], while models sometimes do just as well on instructions that are irrelevant or actively misleading [8]. Wording matters, and reading the wording will not tell you how. That is when you run an experiment.

1.2 Objective and contribution

This project stops treating the instruction file as prose and treats it as a configuration vector. Five components of `program.md` are each written at two levels. All $2^5 = 32$ combinations are generated and run against a real executor, which separates the effect of each component instead of assuming it. The contribution is a method, not an architecture. I do not offer a better autoresearch loop. I offer a way to find out what an existing one is responding to.

The scope is narrow on purpose. Nothing here compares the loop to a human researcher. There is no human baseline and no external standard of good research practice. Every comparison is between two configurations of the same system.

RQ1 Which components of `program.md` causally determine which aspects of the loop's research *process*, and how large is each effect?

RQ2 Do the components act independently of one another, or does the effect of one depend on the level of another?

Section 2 sets out the loop and its configuration space. Section 3 covers the design, the results and what they mean. Section 4 closes.

2 Background

2.1 The loop under study

The loop I study follows the protocol above with one narrowing: the search space is a hyperparameter space, not arbitrary code, so that the instruction file stays the only free variable. The file arrives once as the system message and never changes after that. The model then alternates between two moves, each a single JSON object. A `propose` carries a hypothesis, a model family and hyperparameter values. An `interpret` carries a reading of the last measurement and a decision to continue or stop.

The harness does the executing. It validates each proposal against a fixed whitelist and fits the estimator with scikit-learn. The model never computes a number. It picks what gets computed and reads the answer back. Every value in a transcript is thus traceable: it came from the executor, making it a fact about the task, or from the model, making it a fact about the loop's behaviour. Figure 1 shows the control flow.

2.2 The instruction file as a configuration

Every instruction file comes out of one template. The fixed part gives the research question, the executor interface and the JSON response format, and is byte-identical across all thirty-two files. The variable part is five paragraphs, each written at one of two levels. Those five slots are the independent variables. Nothing else in the file moves. A file is then described completely by

$$c = (c_1, \ldots, c_5), \qquad c_i \in \{0, 1\},$$

taking the slots in the order metric, breadth, stopping rule, output format, emphasis. That gives $2^5 = 32$ files.

Table 1 lists the levels. They are not invented for this study. A real `program.md` hardcodes the baseline to beat, says how failures should be handled, fixes a reporting discipline and imposes a parsimony rule [2], and the five components below come from those categories.

A language model writes the ten paragraphs from fixed meta-prompts. They are then frozen and reused word for word. This is a control. Had I written them myself, the wording of every treatment would have been a free parameter tangled up with my own stylistic habits, and no measured effect could be separated from it. The generator is a different model from the agent. Each paragraph is checked to entail

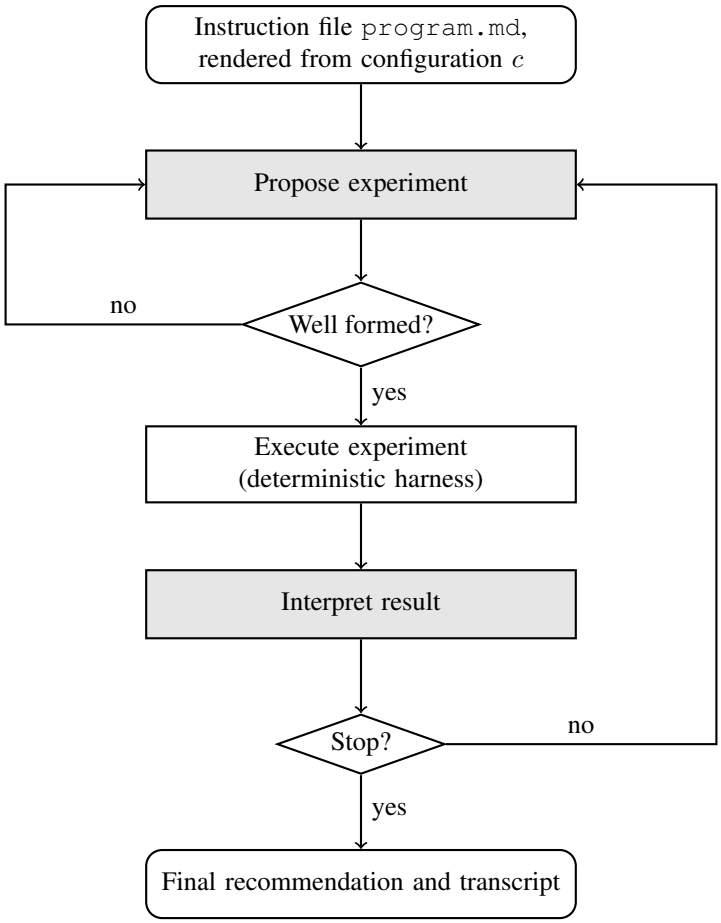

Figure 1: Control flow of one run. Shaded boxes are language-model calls. The unshaded box is deterministic, so every number the loop reasons about was measured and not generated. A malformed or rejected proposal goes back to the proposal step without spending an experiment. The loop halts on the model's decision or on the harness budget, whichever comes first.

its intended level and to say nothing about any other slot, and the two levels of a slot are matched to within ten per cent in token count.

2.3 Configuration change as intervention

I construct the configurations. For each of the thirty-two values of c I generate the file and run it. In observational prompt data the components would correlate, since whoever writes a precise success criterion probably also writes a careful stopping rule. Construction breaks that correlation. Across the thirty-two files each component sits at level 1 in exactly sixteen, and inside each half the other four stay balanced. The difference in mean outcome between runs with $c_i = 1$ and runs with $c_i = 0$ cannot then be produced by any other component. Confounding goes away by intervention, without any statistical adjustment.

Running the whole space buys something beyond the five main effects. With all thirty-two files in hand, every pairwise interaction is estimated separately from every main effect, so whether the components act independently becomes a question I answer from the data. A design that visits only part of the space has to assume an answer, because the runs needed to tell a main effect from an interaction are the ones it skipped. Thirty-two cells is the price of not assuming. At this scale I can pay it.

Table 1: The five varied components of `program.md`. A configuration is one choice of level per row.

Code	Component	Level 0	Level 1
M	Evaluation criterion	Success left undefined; the loop judges quality as it sees fit.	Success defined as five-fold cross-validated accuracy, with the across-fold standard deviation used only to break ties.
B	Search breadth	Commit to one model family for the session.	Compare both families within a session and vary several hyperparameters at once.
S	Stopping rule	Fixed budget of exactly three experiments.	Adaptive budget of up to eight, halted when an experiment fails to improve by more than 0.002.
O	Reporting style	Terse: the action taken and the numbers involved.	Full sentences stating hypothesis, expectation and interpretation.
E	Search emphasis	Explore first: several different configurations before refining any.	Exploit first: refine a promising configuration immediately.

3 Numerical Evaluation

3.1 Experimental setup

Tasks and executor. Every configuration runs against two canonical tabular classification datasets, picked to differ in size, dimensionality and class count: Breast Cancer Wisconsin (569×30, two classes) and Wine (178×13, three classes). Each uses one stratified $70/30$ partition, fixed with the same seed everywhere. The loop may reach for `RandomForestClassifier` or `GradientBoostingClassifier` under a fixed hyperparameter whitelist. Whatever a configuration's evaluation paragraph stresses, the executor returns the same record every time: five-fold cross-validated accuracy, its across-fold standard deviation, and held-out validation accuracy. The instruction file changes what the loop attends to. It never changes what gets measured.

Baseline. One reference quantity is computed per dataset before any run, by a procedure that involves no language model. $a_0(d)$ is the accuracy of a default-hyperparameter fit, taken as the better of the two model families. Every run is measured against it. No exhaustive grid search is needed, which keeps the reference cheap and keeps an estimated quantity out of the measurement.

One task limitation follows. The second dataset's 54-instance validation split is classified perfectly by default hyperparameters, so a_0 is 1.0 there and no run can improve on it. On that task I measure the gain on the cross-validated scale only. That is a property of the task, not of the metric.

Control of non-treatment variation. These are held fixed and so cannot explain any measured effect: the agent model, pinned to the dated snapshot `gpt-4o-mini-2024-07-18` instead of a moving alias; the decoding temperature, one value for every run; the partition and its seed; the executor and its whitelist; the fixed part of the template; the response format; and the harness safety caps. Runs are separate conversations, so nothing carries over between them.

Sampling noise is different. I control it instead of removing it. Every run passes an explicit sampling seed to the model interface, so any run can be regenerated exactly, and every cell draws from the same seed list, so the draws cannot favour one configuration. Greedy decoding at temperature zero would be the wrong move here. Run-to-run variance is the error term that every coefficient below is tested against, and a deterministic loop drives it to zero, taking every standard error with it. I hold temperature constant and above zero, which makes it a controlled constant. When a run fails to produce a valid experiment

it is repeated with a fresh seed until the cell holds exactly R completed runs, so balance survives by construction.

Design and budget. The design is the complete 2^5 factorial crossed with the task suite and replicated R times, giving $N = 2^5 \cdot D \cdot R$ runs. I set $D = 2$ and $R = 20$, so $N = 1280$. That replicate count is not convenience. It fixes the smallest detectable level switch at $0.22\,\sigma$ per dataset, where σ is the run-to-run standard deviation of whichever metric is being fitted.

3.2 Performance metrics

Metrics come straight off the structured transcript. No evaluation framework, scoring library or judge model is involved, because every metric is a difference in accuracy, a share of the proposals, or a count of executor calls.

Measurement window. The stopping rule sets how many experiments a run performs, and every metric below aggregates over those experiments. A best score improves with the number of trials for reasons that have nothing to do with how well the loop searched, and a ratio taken over more proposals is estimated more finely. Every metric is computed over the first three experiments, the number the fixed-budget level performs and so the number every run has. Complete-run values appear separately.

Outcome metrics. The gain over default is the difference between the score of the configuration the run finally recommends and $a_0(d)$. It answers the only question that matters about the result: did the loop improve anything. A negative value means the run ended on a configuration worse than untuned defaults.

The improvement rate is the share of executed trials whose score is strictly higher than the best score seen earlier in the same run. A run of eight trials with one improvement is close to guessing. A high rate is evidence that the loop is using the interpret step and not proposing at random.

Efficiency metrics. The wasted trial ratio is the number of proposals rejected before execution, whether malformed or invalid against the whitelist, plus the number that exactly duplicate an earlier proposal in the same run, divided by the total number of proposals attempted. Both failure modes burn a turn without buying a measurement. A high ratio means the loop is either fighting the response format or not reading its own transcript.

The cost to best is the number of executor calls made up to and including the one that produced the run's highest score, taking the earliest such call on ties. Exploration after that point does not count. Every executed trial costs the same fixed budget, so this is the compute spent to reach the result.

One thing to watch: the evaluation-criterion component at level 1 names cross-validated accuracy, which is the scale the gain over default is computed on. Part of any effect that component shows is the treatment agreeing with the instrument. On the first dataset I report the gain on held-out validation accuracy as a robustness check and claim an effect for that component only where both scales agree. The second dataset admits no such check, for the reason given in Section 3.1.

3.3 Regression model and inference

Code each component as an indicator variable

$$X_i = \begin{cases} +1 & \text{if component } i \text{ is at level } 1, \\ -1 & \text{if at level } 0, \end{cases} \qquad i = 1, \ldots, 5, \tag{1}$$

Table 2: Main-effect estimates $2\hat{\beta}_i$ per metric from equation (2), with heteroskedasticity-consistent standard errors and Benjamini–Hochberg adjusted p-values. The $\hat{\gamma}$ column is the fitted dataset fixed effect (Wine relative to Breast Cancer). **[PLACEHOLDER: populate from `regression_main.csv`.]**

Metric	M	B	S	O	E	$\hat{\gamma}$	R^2
Gain over default	–	–	–	–	–	–	–
Wasted trial ratio	–	–	–	–	–	–	–
Improvement rate	–	–	–	–	–	–	–
Cost to best	–	–	–	–	–	–	–

and model the outcome of one run as

$$Y \;=\; \beta_0 + \sum_{i=1}^{5} \beta_i X_i + \sum_{j=2}^{D} \gamma_j D_j + \varepsilon, \tag{2}$$

where D_j indicates that the run was made on dataset j, the first dataset serving as the reference level. The γ_j are dataset fixed effects: they absorb the inherent difficulty of each task, its headroom above $a_0(d)$ and its response to the whitelist, which would otherwise sit in the residual and inflate every standard error. Because the factorial is crossed with the task suite, each X_i is orthogonal to every D_j by construction, so the $\hat{\beta}_i$ are within-task effects of the architectural components and are not contaminated by which tasks happen to be easy. Here β_0 is the mean across configurations on the reference dataset, β_i the effect of component i, and ε run-to-run error. Each X_i takes each value equally often and independently of the rest, so $\hat{\beta}_i$ does not shift when another component enters or leaves the model, and $2\beta_i$ is the average change in Y from switching component i. Running all thirty-two configurations buys the interaction model for free,

$$Y \;=\; \beta_0 + \sum_{i=1}^{5} \beta_i X_i + \sum_{1 \le i < j \le 5} \beta_{ij} X_i X_j + \sum_{j=2}^{D} \gamma_j D_j + \varepsilon, \tag{3}$$

which is how RQ2 gets tested.

I fit by ordinary least squares on the individual runs, not the cell means. Balance means the coefficients come out the same either way, while the error term now reflects real run-to-run scatter. Balance would also equalise the standard errors if the error variance were constant, and it is not: the stopping rule pins the experiment count at one level and frees it at the other, so within-cell variance differs systematically between the two halves of the design. I use heteroskedasticity-consistent standard errors throughout, and fit the wasted trial ratio and the improvement rate on their natural scale, both being proportions. Benjamini–Hochberg adjustment handles multiplicity, with the family taken as the treatment coefficients of one model on one metric.

3.4 Results

Table 2 gives the main-effect estimates and Figure 2 plots them with their intervals. **[PLACEHOLDER: for each of the four metrics, say which component dominates and by how much, and whether the gain over default and the cost to best point at the same components.]**

Table 3 gives the two-way interactions. **[PLACEHOLDER: say whether the components act independently, and name any pair whose interaction rivals a main effect in size.]**

An instruction is not always obeyed. Table 3 therefore also reports the share of runs whose behaviour actually matched the instruction given, so I can tell a component that had no effect from one the model ignored. Three components constrain an observable action and admit a mechanical check: B by the number of model families the run proposes, S by the number of experiments it performs, E by whether

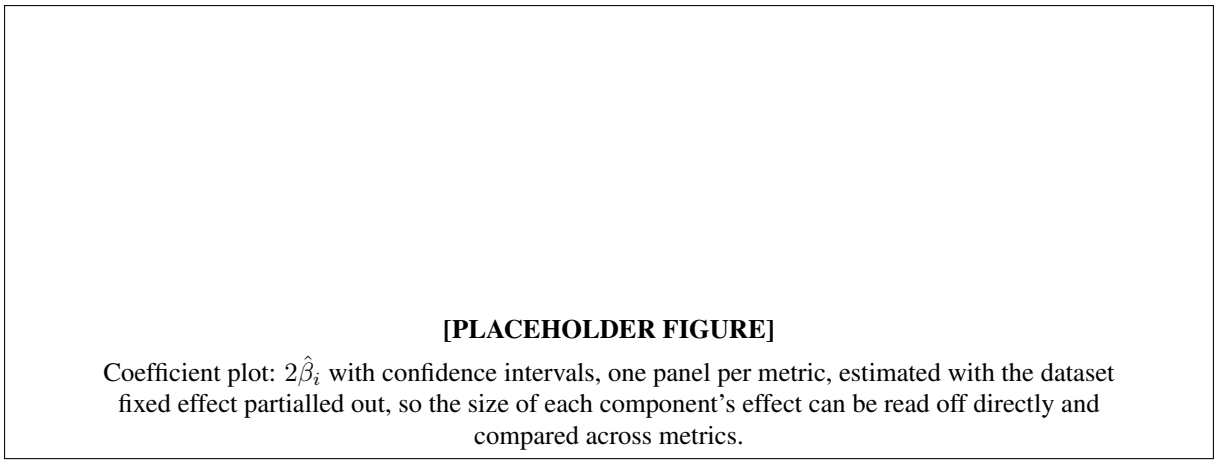

Figure 2: Estimated component effects on every metric. **[PLACEHOLDER.]**

Table 3: Two-way interaction estimates $2\hat{\beta}_{ij}$ from equation (3), and compliance rates per component. **[PLACEHOLDER.]**

Interaction	Estimate	adj. p	Component	Compliance
$M \times B$	–	–	B	–
$M \times S$	–	–	S	–
$B \times S$	–	–	E	–
$S \times E$	–	–	O	– / – chars
...	–	–	M	not observable

its second proposal refines the first. O constrains wording rather than action, so it gets a manipulation check instead, the mean characters of hypothesis and interpretation text per run at each level. M leaves no behavioural trace at all, because the executor computes the score whatever the instruction says. **[PLACEHOLDER.]**

3.5 Discussion and further considerations

[PLACEHOLDER: read the recovered effect structure. Say which components govern the gain over default, the wasted trial ratio, the improvement rate and the cost to best, how large each effect is, and whether any pair of components interacts strongly enough to matter.]

Three limitations bound what any of this shows. Everything is conditional on one agent model, so I have measured how instructions act on one policy, not a property of instructions at large. Each level is realised by a single generated paragraph, which leaves the effect of a component's level tangled with the effect of its particular wording; read the results as effects of specific instruction texts. And the tasks are tabular classification problems over a small discrete hyperparameter space. Whether the same effect structure appears in loops that search over code, or over open-ended hypotheses, is outside what this design can reach.

4 Conclusion

In an autoresearch loop, `program.md` does the job that controller code does in ordinary software, but you cannot read it the way you read code. So I treated it as a configuration vector. Five components of `program.md` were written at two levels each, all $2^5 = 32$ configurations were generated and run against a real executor on two datasets with twenty replicates apiece, and the resulting behaviour was reduced to four outcome metrics and regressed on an orthogonal ± 1 coding of the components. Because

I built the configurations instead of collecting them, the coefficients can be read causally. Because the factorial is complete, whether the components interact is something I can test rather than assume.

[PLACEHOLDER: one paragraph on which components drove which behaviour, how large those effects were, and whether the components turned out to act independently.]

The method is not tied to the five components I happened to vary. Any instruction file that can be cut into slots can be studied the same way, and full enumeration stays cheap while the number of slots is small. Two follow-ups are worth doing. Sampling several wordings per level would separate a component from its phrasing, which is the sharpest limitation above. Running the procedure on a loop that edits code, not hyperparameters, would test it where the instruction file carries far more of the system's behaviour.

References

[1] T. Lütke. Performance: 53% faster parse+render, 61% fewer allocations. Pull request #2056, Shopify/liquid, 2026. Produced by an autoresearch run of approximately 120 experiments over 93 commits; unmerged at the time of writing. `https://github.com/Shopify/liquid/pull/2056`.

[2] B. Tuychiev. A guide to Andrej Karpathy's AutoResearch: automating ML with AI agents. DataCamp tutorial, 23 March 2026. Secondary source; it reports the contents of the repository's `program.md` and results log. `https://www.datacamp.com/tutorial/guide-to-autoresearch`.

[3] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes. Autonomous chemical research with large language models. Nature, 624(7992):570–578, 2023.

[4] Q. Huang, J. Vora, P. Liang, and J. Leskovec. MLAgentBench: Evaluating language agents on machine learning experimentation. ICML, 2024. arXiv:2310.03302.

[5] A. Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. GitHub repository, 2026. `https://github.com/karpathy/autoresearch`.

[6] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The AI Scientist: Towards fully automated open-ended scientific discovery, 2024. arXiv:2408.06292.

[7] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr. Quantifying language models' sensitivity to spurious features in prompt design, or: How I learned to start worrying about prompt formatting. ICLR, 2024. arXiv:2310.11324.

[8] A. Webson and E. Pavlick. Do prompt-based models really understand the meaning of their prompts? NAACL-HLT, pages 2300–2344, 2022.